Este es un ejercicio simple y estructurado que utiliza **JavaScript**, el framework de testing **Jest**, y **GitHub** para simular un ciclo de desarrollo en equipo de tres alumnos (Alfa, Beta, y Gamma), centrado en la detección y solución de errores mediante testing.

El ejercicio se basará en la creación de una utilidad de calculadora simple.

1.Ejercicio: Desarrollo de una Utilidad de Calculadora con Testing1. Herramientas y Frameworks

- **Lenguaje de Programación:** JavaScript (Node.js).
- **Framework de Testing:** Jest (fácil de configurar y usar para pruebas unitarias).
- **Control de Versiones:** GitHub (usando *feature branches* y *Pull Requests*).

2. Flujo de Trabajo en Equipo (GitHub)

Cada paso de desarrollo debe seguir este ciclo, garantizando que los errores y soluciones sean visibles y comentables:

1. Crear una rama nueva (`git checkout -b nombre-de-rama`).
2. Hacer los cambios y *commits*.
3. Subir la rama a GitHub (`git push`).
4. Crear un *Pull Request* (PR) contra la rama principal (`main` o `master`).
5. **Comentar Errores Detectables:** El revisor debe dejar comentarios en el PR sobre el código, especialmente si detecta el potencial error o si el código es difícil de testear.
6. Mergear la rama (solo si pasa la revisión y, al final, los tests).

3. Fases del Ejercicio y Asignación de Roles

Fase 1: Configuración y Desarrollo Inicial (Alfa)

Objetivo: Inicializar el proyecto y la infraestructura de testing, implementando la primera función correcta.

Tarea	Rol
Configuración del Repo	Alfa
Setup de Node.js y Jest: Crear `package.json`, instalar `jest`.	Alfa
Crear Archivos	Alfa
Implementar `add(a, b)`: Una función que suma dos números.	Alfa
Crear Prueba Inicial: Escribir una prueba unitaria exitosa para `add(a, b)`.	Alfa
PR y Revisión	Alfa crea un PR; Beta y Gamma revisan y …

Archivos clave:

- `calculator.js`
- `calculator.test.js`

Fase 2: Implementación e Introducción del Error (Beta)

Objetivo: Desarrollar una nueva función de forma intencionalmente errónea para que el testing la detecte.

Tarea	Rol
Implementar `subtract(a, b)`: Una función que resta dos números.	Beta
Introducir el Error	Beta implementa un *bug* sutil, por ejemplo, en lugar de `return a - b;`, usa `return a - b + 0.0001;`.
PR y Comentarios	Beta crea un PR. **Comenta** el código explicando (sin revelar el bug) su lógica de implementación, o plantea una pregunta sobre cómo validar la precisión de punto flotante, sugiriendo un posible área de riesgo.

(Nota: Beta no debe escribir pruebas para su función, dejando el trabajo de detección a Gamma.)

Fase 3: Testing, Detección y Solución (Gamma)

Objetivo: Actuar como el tester y debugger principal. Escribir pruebas, detectar la falla introducida por Beta, y corregirla.

Tarea	Rol
Revisión del Código	Gamma revisa el PR de Beta (sin mergear).
Escribir Pruebas	Gamma crea una nueva rama (`fix/detect-bug`). En `calculator.test.js`, añade pruebas unitarias exhaustivas para la función

	`subtract(a, b)`.
Practicar el Testing	Gamma ejecuta los tests (`npm test`). **La prueba de `subtract` debe fallar.**
Detección del Error	Analizar el resultado del test fallido (Jest mostrará el valor esperado vs. el valor recibido) para localizar la línea con el error en `calculator.js`.
Solución	Gamma corrige la implementación de `subtract(a, b)` eliminando el error.
PR de Solución	Gamma crea un PR con la corrección, **comentando** el error que encontró y cómo la prueba unitaria lo detectó y verificó su solución.

Ejemplo de Código (El error de Beta)

`calculator.js` (Con el error intencional de Beta)

```
export function add(a, b) {
   return a + b; // Implementación correcta de Alfa
}

export function subtract(a, b) {
   // ERROR INTENCIONAL de Beta:
   return a - b + 0.0001;
}
export function calcularPrecio(precio) {
 if (precio >= 1000) {
          // Corrección: se agrega el signo '='
          return precio * 0.9;
 }
 return precio; }
```

`calculator.test.js` (La prueba que escribe Gamma y que fallará)

```
import { add, subtract } from './calculator';

test('Alfa: add(1, 2) should return 3', () => {
  expect(add(1, 2)).toBe(3);
});

// Prueba escrita por Gamma para detectar el error
test('Gamma: subtract(5, 2) should return 3', () => {
  // Jest mostrará: Expected 3, Received 3.0001
  expect(subtract(5, 2)).toBe(3);
```

```
});
```

La integración de **GitHub Actions** es un excelente paso para automatizar el ciclo de desarrollo y garantizar que solo el código que pasa las pruebas pueda ser fusionado (Merge). Esto refuerza el rol de Gamma y añade una capa de seguridad al proceso de *Pull Request* (PR).

A continuación, se detalla cómo configurar los **GitHub Actions** para el *testing* automático con Node.js y Jest, además de los *tips* para el ejercicio

4. Profundización: Integración de Testing Automático con GitHub ActionsTips Adicionales para el Ejercicio

1. **Protección de la Rama `main`:** Configuren la rama `main` para que solo se pueda fusionar si las **GitHub Actions** han pasado exitosamente. Esto obliga a los alumnos a esperar el resultado del test automatizado antes de aprobar un PR.
2. **Comentarios en el PR:** Cuando se crea un PR, la Acción de GitHub se ejecuta. Los alumnos (especialmente Gamma) deben revisar el *check* que aparece al final del PR. Si el test falla (como en la Fase 2 de Beta), el error de Jest será visible directamente en el log de la Acción, facilitando la práctica de *debugging* remoto.

Pasos para la Configuración del Workflow

Para configurar la automatización, solo necesitan crear un archivo YAML dentro de la carpeta `.github/workflows/` de su repositorio.

1. Crear la Estructura de Carpetas:

En el directorio raíz del proyecto, Alfa o Beta debe crear la siguiente ruta:
.github/workflows/

2. Crear el Archivo de Workflow:

Dentro de esa carpeta, creen un archivo llamado `ci-testing.yml` (el nombre es arbitrario, pero debe tener la extensión `.yml` o `.yaml`).

3. Contenido del Archivo `ci-testing.yml` (JavaScript con Jest):

Este script de GitHub Action configura el entorno de Node.js, instala las dependencias (`npm install`) y ejecuta el comando de pruebas (`npm test`).

```
# .github/workflows/ci-testing.yml
```

Archivo: .github/workflows/ci-testing.yml

```
name: Node.js CI Testing

# Esta acción se ejecutará cada vez que haya un push o un pull request (PR)
# contra las ramas especificadas (en este caso, 'main' y 'master').
on:
  push:
    branches: [ main, master ]
  pull_request:
```

```
    branches: [ main, master ]

jobs:
  build:
    # Se ejecuta en el sistema operativo Ubuntu más reciente de GitHub
    runs-on: ubuntu-latest

    strategy:
      matrix:
        # Se puede especificar qué versiones de Node.js probar.
        # Para este ejercicio, con una es suficiente.
        node-version: [18.x]

    steps:
    # Paso 1: Checkout del Código
    # Obtiene una copia del código del repositorio en la máquina virtual.
    - uses: actions/checkout@v4

    # Paso 2: Configuración de Node.js
    # Configura el entorno de Node.js con la versión especificada.
    - name: Use Node.js ${{ matrix.node-version }}
      uses: actions/setup-node@v4
      with:
        node-version: ${{ matrix.node-version }}
        cache: 'npm' # Habilita el caching para acelerar las instalaciones

    # Paso 3: Instalación de Dependencias
    # Ejecuta el comando para instalar Jest y otras dependencias del package.json.
    - name: Install dependencies
```

```
    run: npm install

  # Paso 4: Ejecución de las Pruebas
  # Ejecuta el script 'test' definido en el package.json (Jest).
  # Si alguna prueba falla (como la de 'subtract' de Beta), esta Acción fallará.
  - name: Run tests
    run: npm test
```

…Fin del archivo : .github/workflows/ci-testing.yml

Nota sobre `package.json`: Para que el Paso 4 funcione, asegúrense de que el archivo `package.json` tenga el script de prueba configurado correctamente, como se presupone en el ejercicio:

```
// En package.json
"scripts": {
  "test": "jest"
}
```

Integración en las Fases del Ejercicio

La adición de GitHub Actions modificaría las fases de la siguiente manera:

- **Fase 1 (Alfa):** Además de configurar Jest, Alfa debe crear la estructura `.github/workflows/` e incluir el archivo `ci-testing.yml` en su PR inicial.
- **Fase 2 (Beta):** Cuando Beta sube el código con el error y crea el PR, la GitHub Action se ejecutará automáticamente. El test fallará y **el PR quedará marcado como fallido**.
    - *Comentario detectable:* Beta ahora tiene una prueba visual (el *check* rojo de la Acción) que le indica que introdujo un error, aunque no sepa dónde.
- **Fase 3 (Gamma):**
    - Gamma revisa el PR de Beta y ve que el *check* de la GitHub Action falló.

- Gamma examina los logs de la Acción para ver la salida de Jest, lo que le mostrará el valor `Expected 3, Received 3.0001` y la línea de código donde ocurrió el fallo.
- Gamma aplica la corrección, sube el cambio a su propia rama de solución, y el PR de Gamma (una vez que la Action pasa) valida que el error ha sido resuelto.

5. Tarea Final.

probar la CI , haciendo funciones simples con error , pero cambiandolos roles de los integrantes del equipo(alfa como beta y gamma, beta como alfa y gamma y gamma como alfa y beta). Agregando archivos y/o funciones para poder hacer lo siguiente.

A. testing y detección de errores .
B. modificaciones y soluciones
C. Utilizar github para la CI